

AIWear接口自动化测试报告

测试人员：本人

测试时间：2026 年 5 月 22 日

测试项目：AIWear 系统

测试类型：接口自动化测试

测试方式：Python + pytest + requests 自动化测试

一、测试概述

本报告用于记录 AIWear 系统接口自动化测试的设计、执行和结果。报告内容仅覆盖接口层验证，不展开项目背景、页面功能说明、UI 自动化测试或性能测试内容。

本次接口自动化测试基于已完成的 `api_auto_test` 测试工程和 Allure 执行结果编写，重点验证后端核心接口在当前测试环境下是否能够按照预期返回统一响应结构、正确处理鉴权、完成业务参数校验，并保证用户数据隔离。

二、测试目标

本次接口自动化测试目标如下：

- 验证 AIWear 后端核心接口的正常业务流程是否可用。
- 验证接口统一返回结构是否符合当前实现约定。
- 验证登录、鉴权、token 缺失、token 格式错误等安全相关场景。
- 验证图片上传、图片查询、图片搜索、图片编辑、图片合并和历史记录查询等核心接口链路。
- 验证参数为空、字段缺失、非法参数、跨用户数据访问等异常场景。
- 通过自动化方式沉淀可重复执行的接口回归测试资产。

三、测试范围

本次接口自动化测试覆盖以下接口模块：

模块	接口或能力	覆盖重点
发送验证码	/api/user/send-code	正常发送、邮箱为空、字段缺失、格式错误、重复发送
用户认证	/api/user/auth	账号密码登录、邮箱验证码登录、错误账号密码、字段缺失、新账号创建
图片上传	/api/file/upload/image	人物图上传、服装图上传、空文件、缺少文件、非法文件、鉴权异常
我的图片列表	/api/file/my-images	当前用户图片列表、用户数据隔离、鉴权异常
图片搜索	/api/file/search	文搜图、图搜图、空关键词、缺少文件、非法鉴权
图片编辑	/api/file/edit	正常编辑、缺少图片、缺少指令、非法 URL、跨用户图片、鉴权异常
图片合并	/api/file/merge	正常合并、缺少图片、缺少指令、跨用户图片、混合用户图片、鉴权异常
历史记录	/api/record/my	查询全部记录、按类型查询、空记录、用户隔离、鉴权异常
退出登录	/api/user/logout	正常登出、缺少鉴权头、格式错误、非法 token、重复登出

不在本报告范围内的内容：

- 1. Web 页面手工功能测试。
- 2. UI 自动化测试执行结果。
- 3. 接口性能、并发能力和响应时间压测结论。
- 4. 项目总体验收结论。

四、测试环境

项目	内容
测试环境地址	http://139.199.222.234
测试工程路径	接口自动化测试/api_auto_test
测试配置文件	config/env.yaml、config/accounts.yaml
测试数据文件	data/user.yaml、data/file.yaml、data/record.yaml
返回结构校验	schema/common_result.json、schema/user_schema.json、schema/file_schema.json、schema/record_schema.json
测试素材	testdata/images 及工作区中已有图片素材
报告结果目录	report/allure-results、report/allure-report
java后端服务环境	Java 后端服务部署于当前测试服务器，负责用户认证、图片管理、图片搜索、图片编辑、图片合并、历史记录查询和退出登录等接口能力。
python AI 服务环境	Python AI 服务部署于当前测试环境，主要支撑图片识别、图片编辑、图片合并等 AI 图片处理相关接口。
依赖组件	MySQL：用于存储用户、图片、编辑记录、合并记录等业务数据。 Redis：用于验证码、token 等登录鉴权相关数据。 OSS：用于存储上传图片、处理后图片及接口返回的图片访问地址。

测试执行依赖当前部署环境中的 Java 后端服务、Python AI 服务、MySQL、Redis 和 OSS 相关能力。其中 Redis 主要用于验证码、token 等登录鉴权相关场景。

五、测试工具与技术

工具或技术	用途
Python	编写和运行接口自动化测试脚本
pytest	组织测试用例、执行测试、控制用例顺序和参数化
requests	发送 HTTP 接口请求
YAML	管理接口测试数据和账号配置
jsonschema	校验接口返回结构
Allure	生成接口自动化测试报告
pytest-order	控制存在业务依赖的接口执行顺序

测试工程采用数据驱动方式组织用例，将接口请求数据、预期状态码、预期消息和关键字段校验内容放在 YAML 文件中，测试脚本负责读取数据、构造请求、执行断言并输出 Allure 结果。

六、测试用例设计

6.1、主要设计

本次接口自动化测试共执行 75 条用例，全部执行通过。

接口	用例分组	用例数量	主要验证内容
/api/user/send-code	TestSendCode	5	邮箱验证码发送相关场景
/api/user/auth	TestAuth	11	用户认证、账号密码登录、验证码登录、新账号创建和异常登录
/api/file/upload/image	TestUploadImage	10	图片上传成功、失败、鉴权和文件合法性
/api/file/my-images	TestMyImages	5	我的图片查询、用户隔离和鉴权异常
/api/file/search	TestSearch	9	文搜图、图搜图和搜索参数异常
/api/file/edit	TestEdit	9	图片编辑成功、参数异常、跨用户访问和鉴权异常
/api/file/merge	TestMerge	10	图片合并成功、参数异常、用户隔离和鉴权异常
/api/record/my	TestRecord	8	历史记录查询、按类型查询、空记录和用户隔离
/api/user/logout	TestLogout	8	正常登出、异常 token、缺少请求头和重复登出
合计	合计	75	核心接口正向、反向和用户隔离场景

6.2、用例设计重点包括：

1. 正向流程：验证登录、上传、搜索、编辑、合并、记录查询等核心接口可正常完成。
2. 参数校验：覆盖空值、字段缺失、非法 URL、非法文件等异常输入。
3. 鉴权校验：覆盖缺少 Authorization、格式错误、非法 token、失效 token 等场景。
4. 用户隔离：验证用户只能访问和处理自己的图片、编辑记录和合并记录。

5. 业务链路：通过前置上传和前置记录准备，验证编辑、合并、记录查询等依赖型接口。

6.3、/api/file/edit 测试用例 (其他测试用例,详见附件)



七、测试脚本设计与实现

接口自动化工程主要由公共封装、配置、测试数据、schema 校验和测试用例组成。

简化后的项目框架如下：

代码块

```
1  api_auto_test/
2  |— pytest.ini          # pytest 执行配置和 Allure 输出配置
3  |— conftest.py         # 公共 fixture 和测试前置数据准备
4  |— README.md           # 接口自动化工程说明
5  |— requirements.txt    # Python 依赖清单
6  |— common/             # 请求封装、断言、日志、YAML 读取等公共方法
7  |— config/             # 测试环境、账号、Redis 等配置
8  |— data/               # YAML 接口测试数据
9  |— schema/             # 接口返回结构校验文件
10 |— testcases/          # pytest 接口测试用例
11 |— testdata/           # 图片等测试素材
12 |— report/             # Allure 原始结果和静态报告
13 |— logs/               # 接口执行日志
```

关键实现说明如下：

实现点	说明
公共前置数据	通过 session 级 fixture 统一准备配置、测试账号、token、图片上传结果和历史记录前置数据，减少用例之间重复准备。
请求封装	统一封装接口基础地址、请求超时、请求日志和响应日志，让测试用例只关注接口路径、参数和预期结果。
数据驱动	使用 YAML 管理接口输入、预期 code、预期 message、关键字段和异常场景，测试脚本按数据生成参数化用例。
鉴权处理	根据用例中的账号或鉴权模式生成 Authorization 请求头，覆盖正常 token、缺少请求头、格式错误和失效 token 等场景。
断言策略	同时校验业务状态码、响应消息、关键字段、用户隔离结果和 JSON schema，避免只判断 HTTP 请求是否成功。
报告输出	pytest 执行结果写入 Allure 原始结果目录，再生成静态报告用于查看用例状态、耗时和执行明细。

脚本执行流程如下：

1. pytest 根据 `pytest.ini` 发现 `testcases` 目录下的测试用例。
2. `conftest.py` 加载环境配置、账号配置和测试素材路径。
3. 测试开始前通过账号密码登录获取用户身份信息，并优先从 Redis 读取最新 token。
4. 需要图片或记录前置数据用例先执行统一准备逻辑，避免每条用例重复构造相同数据。
5. 测试用例从 YAML 中读取参数，构造请求并调用封装后的请求客户端。
6. 断言接口响应中的 `code`、`message`、关键字段、用户隔离结果和 schema 结构。
7. pytest 将执行结果写入 Allure 原始结果目录，再由 Allure 生成可视化报告。

八、测试执行情况

本次报告依据已完成的接口自动化执行结果编写，未重新执行测试。

8.1、测试执行配置：

代码块

```
1  addopts = -vs --alluredir=report/allure-results --clean-alluredir
2  testpaths = testcases
3  python_files = test_*.py
4  python_classes = Test*
5  python_functions = test_*
```

8.2、Allure 报告生成命令：

代码块

```
1 allure generate report/allure-results -o report/allure-report --clean
```

8.3、执行结果汇总：

指标	结果
用例总数	75
通过数	75
失败数	0
阻塞数	0
跳过数	0
未知状态	0
通过率	100%
执行总耗时	242.854 秒
最短用例耗时	0.013 秒
最长用例耗时	69.382 秒

耗时较长的接口主要集中在历史记录查询、图片合并、图片上传和图片编辑等真实业务链路。

这类接口涉及图片处理、外部服务调用或历史数据查询，耗时高于普通参数校验类接口，符合接口业务特点。

九、测试结果分析

从执行结果看，本次接口自动化测试覆盖的 75 条用例全部通过，说明当前测试环境下后端核心接口在既定测试数据和业务场景中能够按照预期返回结果。

具体分析如下：

1. 用户认证相关接口通过，说明账号密码登录、邮箱验证码相关校验、新账号创建和异常登录处理符合当前预期。
2. 鉴权相关异常场景通过，说明缺少请求头、请求头格式错误、非法 token 和重复登出等场景能够被识别并返回预期结果。
3. 图片上传相关接口通过，说明人物图、服装图上传以及空文件、缺文件、非法内容等异常场景均得到验证。
4. 图片搜索、编辑和合并接口通过，说明 AI 图片处理相关接口在当前测试输入下能够完成正向流程，并能处理主要参数异常。
5. 我的图片和历史记录接口通过，说明当前用户数据查询、记录类型筛选和用户隔离逻辑符合预期。
6. schema 校验通过，说明接口返回结构与测试工程中定义的用户、文件、记录相关返回结构一致。

本次接口自动化测试主要验证功能正确性和接口返回结果，不等同于性能测试。接口响应耗时、并发能力和稳定性结论应以接口性能测试报告为准。

十、缺陷记录

本次接口自动化执行结果中未出现失败、阻塞或跳过用例，未记录由本次自动化执行直接发现的接口缺陷。

编号	所属模块	问题描述	严重程度	当前状态	处理结论
无	无	本次接口自动化测试未发现失败用例	无	已确认	无需处理

未发现缺陷只代表本次自动化用例覆盖范围内未出现失败结果，不代表系统不存在所有潜在问题。

十一、测试结论

本次 AIWear 接口自动化测试共执行 75 条用例，75 条全部通过，通过率为 100%。测试覆盖验证码、认证、图片上传、我的图片、图片搜索、图片编辑、图片合并、历史记录和退出登录等核心接口模块。

根据本次执行结果，可以得出以下结论：

- 1. 当前测试环境下，接口自动化覆盖范围内的后端核心接口功能正确。
- 2. 统一返回结构、关键字段和 schema 校验结果符合预期。
- 3. 登录鉴权、参数异常、用户数据隔离等关键接口风险点已通过自动化用例验证。
- 4. 接口自动化测试结果可作为 AIWear 项目测试报告中的专项测试依据。

综合判断：AIWear 系统接口层在本次自动化测试覆盖范围内满足测试目标。

十二、附件说明

12.1、接口自动化测试源码：

<https://github.com/23YuTingYang/AIWear-api-auto-test>

12.2、Allure测试报告：

<https://23yutingyang.github.io/aiwear-allure-report/>

12.3、接口测试用例文档：



AIWear接口测试用例思维导图.xmind
311.79KB



完整测试用例已整理为说明版文档，详见在线文档附件《AIWear系统接口自动化测试用例》



 AIWear接口测试用例.md

12.4、接口文档：

